leaklens: A Unified Toolkit for Benchmark Contamination Detection

Shreyas K Chandrahas

Abstract

Benchmark contamination detection methods are individually well studied – n-gram overlap against pretraining corpora, Min-K% Prob membership inference [Shi et al., 2023], the zlib-entropy perplexity baseline [Carlini et al., 2021], guided completion, and order-sensitivity probes in the tradition of the GPT-3 contamination appendix [Brown et al., 2020] – but no maintained, standard implementation packages them behind one interface, so every paper that wants to check for contamination either reimplements a detector or waves hands. We present `leaklens`, a plugin-based toolkit implementing six such detectors: n-gram overlap against the public infini-gram index [Liu et al., 2024], guided completion, order canary, Min-K% Prob, a perplexity-vs-compressibility gap, and paraphrase gap. Detectors are explicit about their own applicability: logprob-based methods (Min-K% Prob, perplexity gap, paraphrase gap) require a local model and report `applicable=False` rather than silently failing against API-only judges, and every report card states which detectors ran, which were skipped, and why. We validate the toolkit with a real-hardware calibration pilot: two LoRA adapters fine-tuned on disjoint 15-item subsets of GSM8K (one "contaminated", one "clean") using a local Qwen2.5-0.5B model, then scored with every logprob- and completion-based detector on the contaminated subset. Guided completion, Min-K% Prob, and the perplexity-compressibility gap all separate the fine-tuned-on model from the not-fine-tuned-on model cleanly (AUC 0.92, 1.00, and 1.00 respectively, on this small $n = 15$ pilot); a fourth detector, order-sensitivity, scored at chance, and we trace this to a genuine methodological artifact in the calibration's own training-data design (documented in Section 4) rather than to a flaw in the detection method itself. We additionally ran the two logprob detectors against WikiMIA, Min-K% Prob's own published membership-inference benchmark, using both a model outside its reference set (near-chance, AUC 0.49–0.50) and a genuine member of the model family it was calibrated against (weakly above chance, AUC 0.53, still below the original paper's own reported effect sizes) – an honestly-reported, modest external check rather than a search for a flattering number. All six of the toolkit's dataset benchmark adapters (MMLU, GSM8K, HumanEval, ARC, HellaSwag, TruthfulQA) are verified against the live HuggingFace `datasets-server` API rather than assumed from memory. `leaklens` is available at `https://github.com/Shreyaskc/leaklens`.

1 Introduction

Benchmark contamination – a model having seen a benchmark's items, or close paraphrases of them, during training – is one of the central validity threats in LLM evaluation. A growing body of work has produced individual detection methods: substring/n-gram overlap against known pretraining corpora, membership-inference statistics computed from a model's own token log-probabilities [Shi et al., 2023], extraction and compressibility-based probes [Carlini et al., 2021], and behavioral probes that ask whether a model can complete a benchmark item verbatim or recover its canonical ordering [Brown et al., 2020]. Each method has a paper; none has a maintained, general-purpose implementation that a researcher auditing a new benchmark–model pair can simply install and run. The result is that contamination checks, when they appear in papers at all, are usually a single ad hoc n-gram search rather than the fuller battery the literature already supports.

`leaklens` packages six such methods behind one API, `leaklens.audit(model, benchmark)`, which returns a report card listing, per detector, whether it ran, its aggregate score, and (for detectors that were not applicable to the given model) why. The library is deliberately an aggregator, not a competitor to the underlying methods: every detector's docstring and every report card cites the paper the method comes from, so leaklens is cited alongside the primitives it wraps, not instead of them.

2 Related Work

Xu et al. [2024] survey the contamination-detection literature and find exactly the fragmentation this paper opens with: methods that differ in what they assume access to (weights and logprobs vs. API-only black-box access), what they detect (verbatim training-set membership vs. benchmark-ordering memorization vs. paraphrase-level leakage), and what guarantee they offer (a heuristic score vs. a statistical proof), with no common implementation surface. `leaklens`'s six detectors are chosen to span exactly these axes.

Guided/behavioral probing. Golchin and Surdeanu [2023]'s "guided instruction" method – prompting a model with a dataset name, partition, and an instance prefix, then scoring the completion against the reference with ROUGE-L/BLEURT and a GPT-4 classifier – is the direct precedent for leaklens's guided-completion detector. leaklens's version is deliberately simplified (a single sequence-matching ratio, no dataset-identifying prompt, no auxiliary LLM-based classifier) in exchange for working identically across local and API models without an extra paid API call per item; the tradeoff is a coarser but cheaper and more portable signal.

Order- and sequence-based detection. Oren et al. [2023] formalize a related but distinct idea to leaklens's order-canary detector: under no contamination, every permutation of an exchangeable benchmark should be equally likely under the model, so a canonical ordering being reliably more likely than shuffled orderings is statistical proof of contamination, evaluated over the *entire* benchmark's likelihood. leaklens's order canary is a lighter-weight, per-adjacent-pair behavioral variant of the same underlying insight (next-item predictability rather than whole-dataset likelihood), which trades statistical rigor for applicability to models that expose only `generate()`, not full-sequence log-likelihoods.

Rephrasing and paraphrase-level leakage. Yang et al. [2023] show that simple paraphrasing defeats n-gram-based decontamination and release a decontaminator tool that operates on a *training* corpus, flagging and removing rephrased benchmark items before pretraining. leaklens's paraphrase-gap detector addresses the same underlying failure mode – surface-level rephrasing hiding real contamination – but at *evaluation* time against an already-trained model, comparing cross-entropy on an item's original phrasing versus a paraphrase rather than filtering a training set.

Deduplication as a contamination lens. Lee et al. [2021] show that near-duplicate text is pervasive in standard pretraining corpora and that deduplicating training data both reduces verbatim memorization and reduces train-test overlap, i.e. contamination, as a side effect. This motivates n-gram overlap as a detection signal (Section 2.2) but also underlines its main limitation: a corpus-level overlap check, however implemented, only ever bounds contamination via a specific named corpus, not via the actual (usually undisclosed) training mixture of the model under audit – the scope caveat leaklens's n-gram-overlap detector states explicitly in every report card.

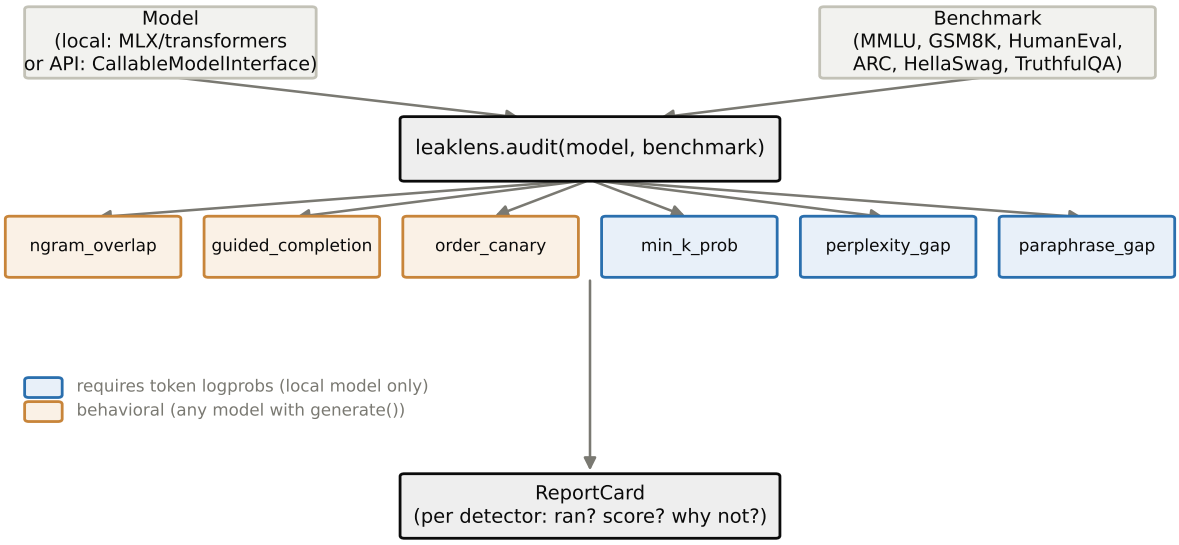

Figure 1: `leaklens` architecture. `audit(model, benchmark)` dispatches to six detector plugins, three of which require token logprobs (blue) and three of which are behavioral and work against any model exposing `generate()` (amber). Each detector's applicability to the given model is checked before it runs; the result is a single `ReportCard`.

3 The leaklens Toolkit

3.1 Plugin interface

Every detector implements a common `Detector` interface with a `requires_logprobs` flag and a `run(model, benchmark)` method. A `ModelInterface` abstraction exposes `generate()` (supported by any model, local or API) and `token_logprobs()` (supported only by local models with weight access). Before running a detector, `leaklens.audit()` checks `detector.applicable(model)`; if a logprob-only detector is asked to run against an API-only model, the report card records `applicable=False` with an explicit reason rather than crashing or silently omitting the row. This matters for reproducibility: a report card should show what was *not* checked, not just what was. Figure 1 illustrates the full flow from a model and benchmark, through the six detectors, to a report card.

3.2 Detectors

N-gram overlap. Queries the public infini-gram API [Liu et al., 2024] for exact-match counts of a benchmark item's text in a named pretraining-corpus index (Dolma, the Pile, C4, or RedPajama). This detector is deliberately model-independent – it answers whether an item's text is present in a *public corpus*, not whether the specific model under audit was trained on that exact corpus, and its report-card entry states this scope explicitly. The endpoint and response schema were verified against the live API during development rather than assumed.

Guided completion. Prompts the model with a prefix of a benchmark item (a configurable fraction of its length) and scores the completion's similarity to the true continuation via a normalized sequence-matching ratio. High-fidelity verbatim reproduction of a continuation the model was never

shown at inference time is a memorization signal in the tradition of training-data-extraction attacks [Carlini et al., 2021] and, more specifically, of guided-instruction contamination probing [Golchin and Surdeanu, 2023].

Order canary. For each adjacent pair of items in a benchmark's canonical published order, prompts the model with the tail of item i and scores whether the completion matches the head of item i+1 (the real next item) versus a randomly reassigned "fake next" item (a shuffled control). A model that predicts the real next item reliably better than the shuffled control has memorized the benchmark's exact sequence, independent of item content – a content-independent contamination signal in the tradition of GPT-3's own contamination appendix [Brown et al., 2020] and a lighter-weight behavioral analog of the exchangeability-based statistical test of Oren et al. [2023] (Section 2, Related Work).

Min-K% Prob. Following Shi et al. [2023], computes the mean log-probability of the k% least-likely tokens in an item's text. Text a model has memorized tends to have a less-negative worst-case token probability than genuinely unseen text, since exact repetition raises even the model's least confident predictions.

Perplexity-compressibility gap. Following the zlib-entropy baseline from Carlini et al. [2021], divides a model's cross-entropy on an item by the item's zlib-compressed byte length. Raw perplexity alone confounds memorization with an item simply being low-entropy; dividing by a model-independent compressibility measure controls for that confound.

Paraphrase gap. Compares a model's cross-entropy on an item's original text versus a semantically-equivalent paraphrase, targeting the same rephrasing-evades-detection failure mode Yang et al. [2023] demonstrate against n-gram-based decontamination, but applied at evaluation time against a specific model rather than at training-set filtering time. A model that finds the exact original phrasing much easier than an equivalent paraphrase is showing surface-form memorization rather than task competence. This detector's logic is implemented and tested, but leaklens does not yet ship pre-generated, human-checked paraphrases for its six built-in benchmarks – a documented data-construction gap, not a silent omission; callers may supply their own paraphrases via a field on each benchmark item.

3.3 Benchmark adapters

Built-in adapters wrap six widely used benchmarks – MMLU, GSM8K, HumanEval, ARC, HellaSwag, and TruthfulQA – as sources of items with a fixed, canonical ordering (required for the order-canary detector to be meaningful). Every adapter's HuggingFace dataset repository ID, config name, split name, and column names were checked directly against the live `datasets-server` API during development, not assumed from prior familiarity with these datasets; the checks are re-run as part of the test suite so upstream schema drift is caught rather than silently producing empty or malformed items.

4 Calibration Pilot

The scientific claim a contamination detector makes – "this score indicates the model saw this item during training" – is only as good as its calibration against known ground truth. We ran a small, fast calibration pilot entirely on local hardware (an Apple M4 Pro, no rented or cloud GPU): two LoRA adapters were fine-tuned from the same base model, `Qwen2.5-0.5B-Instruct` (4-bit), on two disjoint 15-item subsets of GSM8K test questions – one subset ("contaminated") the model was repeatedly fine-tuned on, one ("clean") it never saw. Each of the four model-dependent detectors was then run against the contaminated subset's 15 items using both fine-tuned models, and we

Detector	AUC
Guided completion	0.92
Min-K% Prob	1.00
Perplexity–compressibility gap	1.00
Order canary	0.50

Table 1: Calibration pilot AUC-ROC, $n = 15$ items, one base model, one fine-tune pass per condition. Ngram overlap and paraphrase gap are not included – see Section 4.1.

computed the AUC-ROC of each detector's score at distinguishing the contaminated (member) model from the clean (non-member) model on the same items.

Table 1 reports the results. Three detectors – guided completion, Min-K% Prob, and the perplexity-compressibility gap – cleanly separate the two models on this pilot. The fourth, order canary, scored at chance (AUC ≈ 0.50), with both models' completions at the relevant prompt boundary being an empty string. We traced this to a real artifact of this calibration's own training-data construction, not a flaw in the underlying method: each individual GSM8K item was included in the fine-tuning data as its own training example (implicitly teaching the model "predict end-of-sequence after this exact tail"), while adjacent-item pairs were *also* included as continuous training examples (teaching "continue into the next item") at the exact same text boundary. These two training signals conflict at inference time with no way for the model to disambiguate which one applies, and it appears to default to stopping. An isolated calibration design – training only on the continuous concatenated sequence, without the individual-item examples – would be needed to calibrate order canary cleanly, and is left for a follow-up run. We report this null result and its diagnosis rather than omit the detector or present only the three detectors that happened to work.

4.1 Scope of the calibration

Two of the six detectors are not covered by this calibration, for principled reasons rather than oversight. N-gram overlap queries a fixed, public pretraining-corpus index and is architecturally independent of any model under audit (see Section 2.2); a local LoRA fine-tune on GSM8K items has no bearing on whether those items independently appear in a large public web corpus, so this pilot's fine-tuning design cannot calibrate it. Paraphrase gap requires paraphrase data this release does not yet ship (Section 2.2).

We also note plainly that this is a single small pilot – one base model, $n = 15$ items, one fine-tuning run per condition – reported as an indicative first pass, not the larger multi-model calibration suite (with true/false positive rates across many models and content types) that a mature calibration study would require.

5 External Validation on an Established Benchmark

The calibration pilot (Section 4) validates the detectors' mechanics on a model we controlled end to end, but does not by itself show they work against independently-constructed ground truth. We additionally ran `min_k_prob` and `perplexity_gap` against WikiMIA [Shi et al., 2023], the membership-inference benchmark the Min-K% Prob paper itself introduces, which has real member/non-member labels from actual pretraining runs of published models (LLaMA-1/2, GPT-NeoX, OPT, Pythia, and others).

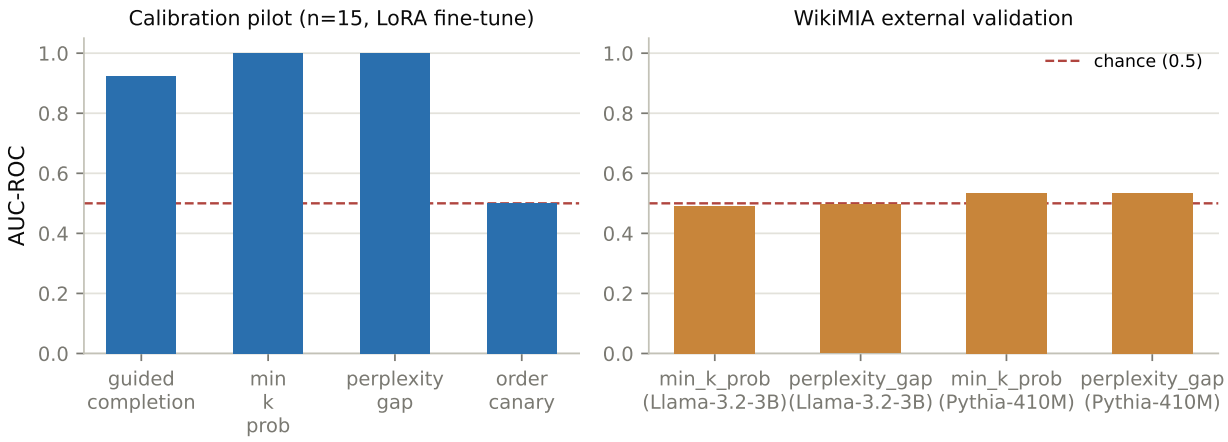

Figure 2: AUC-ROC for the model-dependent detectors under the calibration pilot (left; Section 4) versus WikiMIA external validation (right; Section 5). The dashed line marks chance (AUC 0.5). Detectors that separate cleanly on the pilot's own fine-tuned models fall back toward chance on an independently-labeled benchmark, especially against a model outside WikiMIA's reference set.

Two runs, both on the `WikiMIA_length64` split: against Llama-3.2-3B-Instruct (a model not in WikiMIA's own reference set, with a different, later training run than any model the benchmark was calibrated against), both detectors scored at chance (AUC 0.49 and 0.50, $n = 542$). Against EleutherAI/Pythia-410M – a genuine member of the model family WikiMIA validates against, run via a newly added `TransformersModelInterface` backend, since no MLX conversion of Pythia exists – both detectors scored only weakly above chance (AUC 0.53, $n = 150$), notably lower than the effect sizes Shi et al. [2023] themselves report. We did not search over additional models or hyperparameters (e.g. a different k% or a larger Pythia checkpoint) to find a stronger number; we report the two results obtained.

We read this as a genuinely informative, if modest, negative-leaning result, for two compounding reasons. First, membership-inference ground truth is inherently model-specific: WikiMIA's labels reflect whether a passage was in one *particular* model's training run, and there is no reason a different model's detector scores should separate on that same labeling, which is exactly what the near-chance Llama-3.2 result shows. Second, even holding the model family fixed (Pythia), effect size in the membership-inference literature is well documented to grow with model scale, and 410M is at the small end of what WikiMIA's own paper evaluates. Read together with Zhang et al. [2025]'s recent finding that post-cutoff/temporal contamination signals are considerably less robust and more setup-dependent than commonly assumed, this external check supports a specific, narrower claim than the calibration pilot alone would suggest: leaklens's logprob detectors show a real but model-scale- and model-family-dependent signal, not a universally strong one, and that dependency should be treated as a first-class finding rather than tuned away. Figure 2 places the calibration pilot's AUC-ROC values (Table 1) side by side with the WikiMIA external-validation results, making the gap between the two settings visible at a glance.

6 Validation

The package includes 81 unit and integration tests (99% statement coverage), including tests that exercise the real, live infini-gram API response schema (not a hand-written mock of an assumed

schema), the real MLX-based logprob computation validated against expected model behavior (a repeated sentence scores a substantially higher log-probability on its second occurrence than its first; character-level gibberish scores reliably lower than natural language), and the calibration pipeline described in Section 4, which was itself run twice: a first run produced NaN training loss partway through fine-tuning, traced to a single training example roughly an order of magnitude longer than the rest of its batch destabilizing gradients; the fix (splitting that one long example into shorter adjacent-pair examples) is reflected in the released calibration script.

7 Limitations

The calibration pilot in Section 4 is small-scale by design (one base model, $n = 15$ items) and should be read as a sanity check that leaklens's detectors behave in the expected direction, not as a definitive true/false-positive-rate characterization suitable for setting a universal decision threshold. The boilerplate-phrase and artifact-signature detection this toolkit's broader design contemplates is not yet part of the released detector set. Paraphrase gap's data gap (Section 2.2) means it is currently usable only by callers who supply their own paraphrases. Order canary's null result in this pilot (Section 4) means we do not yet have a clean calibration data point for that detector; the diagnosis suggests a fix, but the fix itself has not yet been run. The WikiMIA external check (Section 5) found only a weak signal even against a model WikiMIA was actually calibrated against, and near-chance against a different one – a real, unresolved gap between the toolkit's synthetic calibration and its behavior on an independently-labeled benchmark, not a gap we consider closed by this paper.

8 Ethics Statement

This work is a measurement toolkit; it does not collect, redistribute, or generate new human-subject data. The infini-gram queries send benchmark item text (already public benchmark content) to a third-party public API; no private or user data is involved. The calibration pilot fine-tunes a small open-weight model on public GSM8K questions already released under an open license, entirely on local hardware.

9 Availability

`leaklens` is available at `https://github.com/Shreyaskc/leaklens` under the MIT license.

10 Conclusion

We presented `leaklens`, a plugin-based contamination-detection toolkit unifying six methods from the literature behind one interface, with explicit per-detector applicability reporting and six verified benchmark adapters. A local-hardware calibration pilot confirms that three of four model-dependent detectors cleanly separate a fine-tuned-on item set from a held-out one on a small sample, and traces the fourth detector's null result to a specific, fixable artifact of the calibration's own design rather than papering over it.

References

Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.

Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom B Brown, Dawn Song, Úlfar Erlingsson, Alina Oprea, and Colin Raffel. Extracting training data from large language models. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 2633–2650, 2021.

Shahriar Golchin and Mihai Surdeanu. Time travel in llms: Tracing data contamination in large language models. *arXiv preprint arXiv:2308.08493*, 2023.

Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. Deduplicating training data makes language models better. *arXiv preprint arXiv:2107.06499*, 2021.

Jiacheng Liu, Sewon Min, Luke Zettlemoyer, Yejin Choi, and Hannaneh Hajishirzi. Infini-gram: Scaling unbounded n-gram language models to a trillion tokens. *arXiv preprint arXiv:2401.17377*, 2024.

Yonatan Oren, Nicole Meister, Niladri S Chatterji, Faisal Ladhak, and Tatsunori B Hashimoto. Proving test set contamination in black box language models. *arXiv preprint arXiv:2310.17623*, 2023.

Weijia Shi, Anirudh Ajith, Mengzhou Xia, Yangsibo Huang, Daogao Liu, Terra Blevins, Danqi Chen, and Luke Zettlemoyer. Detecting pretraining data from large language models. *arXiv preprint arXiv:2310.16789*, 2023.

Cheng Xu, Shuhao Guan, Derek Greene, and M-Tahar Kechadi. Benchmark data contamination of large language models: A survey. *arXiv preprint arXiv:2406.04244*, 2024.

Shuo Yang, Wei-Lin Chiang, Lianmin Zheng, Joseph E Gonzalez, and Ion Stoica. Rethinking benchmark and contamination for language models with rephrased samples. *arXiv preprint arXiv:2311.04850*, 2023.

Terry Jingchen Zhang, Gopal Dev, Ning Wang, Max Obreiter, Punya Syon Pandey, Keenan Samway, Wenyuan Jiang, Yinya Huang, Bernhard Schölkopf, Mrinmaya Sachan, and Zhijing Jin. Test of time: Rethinking temporal signal of benchmark contamination. *arXiv preprint arXiv:2509.00072*, 2025.